

BÁO CÁO KỸ THUẬT

**Nhận dạng cá nhân trong hỗn hợp DNA:
Bộ phân loại đa nhân 45 lớp trên dữ liệu PROVEDIt**

1. Giới thiệu

Phân tích DNA hỗn hợp (DNA mixture) là bài toán quan trọng trong pháp y – khi mẫu vật chứa vật liệu di truyền của nhiều cá nhân, hệ thống cần xác định có bao nhiêu người đóng góp và cụ thể là ai. Các hệ thống truyền thống như STRmix hay EuroForMix dựa trên mô hình xác suất phức tạp và đòi hỏi chuyên gia vận hành. Gần đây, các phương pháp học máy bắt đầu cho thấy tiềm năng tự động hóa quá trình này.

Báo cáo này trình bày phần cá nhân của dự án nhóm: xây dựng bộ phân loại đa nhãn (multi-label classifier) 45 lớp xác định những người đóng góp đã biết trong một hỗn hợp DNA. Với mỗi mẫu đầu vào là vector đặc trưng 252 chiều mô tả chiều cao đỉnh (peak height) tại các vị trí STR loci, đầu ra là vector nhị phân 45 chiều cho biết donor nào có mặt trong hỗn hợp.

Phần OOD detection (phát hiện cá nhân chưa biết) là phần của thành viên khác trong nhóm và sẽ được tích hợp vào pipeline chung ở giai đoạn sau.

2. Định nghĩa bài toán

Cho một mẫu hỗn hợp DNA ký hiệu $x \in \mathbb{R}^{252}$, tìm vector nhãn $y \in \{0,1\}^{45}$ sao cho $y_i = 1$ khi và chỉ khi donor i là một trong những người đóng góp vào hỗn hợp. Đây là bài toán multi-label classification – mỗi mẫu có thể có nhiều nhãn đồng thời (tương ứng với hỗn hợp 2–5 người).

Thiết lập closed-set: toàn bộ quá trình huấn luyện và đánh giá trong báo cáo này chỉ xét các mẫu mà tất cả donor đều nằm trong tập 45 người đã biết. Bộ 5 donor hold-out đóng vai 'unknown' và được xử lý riêng bởi module OOD.

3. Dữ liệu

3.1 Nguồn dữ liệu

Dữ liệu được lấy từ bộ dữ liệu PROVEDIt (Proficiency Review of Evidentiary DNA Interpretation Tools), phiên bản RD14-0003. PROVEDIt là bộ dữ liệu mở do Rutgers University tạo lập, bao gồm hàng nghìn mẫu STR (Short Tandem Repeat) được phân tích dưới nhiều điều kiện thí nghiệm khác nhau (loại máy, bộ kit, nồng độ DNA, điều kiện tiêm).

Phiên bản sử dụng trong dự án này: điều kiện GF (GlobalFiler kit), thời gian tiêm 5 giây, máy 3500 với marker SE33F. Tổng cộng 50 donor duy nhất (IDs 1–50), bao gồm các mẫu đơn (1-person) và hỗn hợp (2–5 người).

3.2 Format file thô

Mỗi file CSV thô chứa kết quả phân tích STR của nhiều mẫu. Mỗi hàng tương ứng với một cặp (mẫu, locus). Các cột chính:

Cột	Kiểu	Mô tả
Sample File	string	Tên file .hid, chứa mã hóa donor ID và điều kiện thí nghiệm

Marker	string	Tên STR locus (D3S1358, vWA, D16S539, ...)
Dye	string	Màu thuốc nhuộm điện di (B, G, Y, R)
Allele 1..100	string	Giá trị allele (số nguyên/thập phân/OL/NaN)
Size 1..100	float	Kích thước fragment (bp)
Height 1..100	float	Chiều cao đỉnh RFU (relative fluorescence unit)

3.3 Quy ước đặt tên file (naming convention)

Tên file mã hóa toàn bộ thông tin cần thiết để xác định donor. Ví dụ:

```
1-person : A02_RD14-0003-15d2U60-0.25GF-Q4.5_01.5sec.hid
           └ donor ID = 15 (hai chữ số đầu trước ký tự chữ)

2-person : A02_RD14-0003-31_32-1;1-M2c-0.03GF-Q2.0_01.5sec.hid
           └ donors = {31, 32} (phân tách bằng dấu _)

5-person : A02_RD14-0003-43_44_45_46_47-1;1;2;1;1-M4c-...
           └ donors = {43, 44, 45, 46, 47}
```

Hàm parse được cài đặt trong data/prepare_data.py:

```
def parse_donors(filename: str) -> list[int]:
    parts = filename.split('-')
    contrib_part = parts[2]          # '31_32' hoặc '15d2U60'
    if '_' in contrib_part:
        return [int(re.match(r'^(\d+)', s).group(1))
                for s in contrib_part.split('_')]
    else:
        m = re.match(r'^(\d+)', contrib_part)
        return [int(m.group(1))] if m else []
```

3.4 Tiền xử lý dữ liệu

Pipeline tiền xử lý kế thừa từ module NOC_DNA (bài toán trước của nhóm), thực hiện các bước sau:

Bước	Mô tả
1. Melt wide → long	Biến ma trận Allele/Size/Height thành dạng long (mỗi peak một hàng)
2. Lọc ngưỡng	Loại bỏ peaks có Height < 50 RFU (nhiều), Allele = OL (off-ladder)
3. Stutter filter	Loại stutter artifact (peak N-1, < 15% chiều cao peak chính)
4. Pivot genotype	Pivot về dạng ma trận thưa: Sample File × (Locus, Allele) → Height
5. Flatten header	Đổi tên cột (Marker, Allele) → 'Marker Allele' (ví dụ: D3S1358 15)

Kết quả: file combined_preprocessed_data.csv với 3.342 mẫu, 252 cột đặc trưng (allele heights theo locus_allele), 1 cột Sample File và 1 cột target_noc (không dùng trong task này).

3.5 Phân chia donor và xây dựng nhãn

50 donor được chia thành 45 known và 5 unknown dùng numpy random seed=42 để đảm bảo tái lập được:

```
rng = np.random.default_rng(42)
_shuffled = rng.permutation(list(range(1, 51))).tolist()
```

```
UNKNOWN_DONORS = sorted(_shuffled[:5])    # [6, 21, 26, 40, 50]
KNOWN_DONORS    = sorted(_shuffled[5:])    # 45 donors còn lại
```

Nhãn mỗi mẫu là vector 45-dim binary: $y[i] = 1$ khi KNOWN_DONORS[i] có mặt trong hỗn hợp. Các mẫu chứa bất kỳ donor unknown nào được tách riêng (open-set, 451 mẫu) để tích hợp OOD sau. Closed-set còn lại: 2.891 mẫu.

Tập	Số mẫu	Tỷ lệ
Train	2.023	70%
Validation	434	15%
Test	434	15%
Open-set (OOD)	451	—

4. Kiến trúc mô hình

4.1 Baseline: XGBoost One-vs-Rest

Baseline sử dụng XGBoost với chiến lược One-vs-Rest (OvR): huấn luyện 45 bộ phân loại nhị phân độc lập, mỗi cái dự đoán sự có mặt của một donor cụ thể. Cách tiếp cận này không khai thác tương quan giữa các donor nhưng cho kết quả mạnh trên tabular data thưa.

```
base = XGBClassifier(
    n_estimators=300, max_depth=6, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8,
    scale_pos_weight=5, # bù class imbalance
    tree_method='hist', random_state=42
)
model = MultiOutputClassifier(base, n_jobs=-1)
model.fit(X_train, y_train)
```

scale_pos_weight=5 đặt trọng số cao hơn cho class dương để bù imbalance (trung bình ~3% sample của mỗi lớp là dương).

4.2 Variant 1: MLP (Multi-Layer Perceptron)

MLP xử lý vector 252-dim trực tiếp qua các lớp fully-connected. Kiến trúc tham khảo từ Phan et al. (2021) – bài báo dùng MLP làm backbone phân loại contributor trên sequencing data, với các lớp hidden có BatchNorm và Dropout để tránh overfitting trên dataset nhỏ.

Lớp	In → Out	Chi tiết
Linear + BN + ReLU + Dropout	252 → 512	dropout=0.3
Linear + BN + ReLU + Dropout	512 → 256	dropout=0.3
Linear (output)	256 → 45	logits, sigmoid ở inference

```
class MLP(nn.Module):
    def __init__(self, in_features=252, hidden_dims=[512,256],
                  n_classes=45, dropout=0.3):
        layers = []
        for h in hidden_dims:
            layers += [nn.Linear(prev,h), nn.BatchNorm1d(h),
                       nn.ReLU(inplace=True), nn.Dropout(dropout)]
        layers.append(nn.Linear(prev, n_classes))
        self.net = nn.Sequential(*layers)
```

4.3 Variant 2: 1D CNN theo locus

Thay vì xử lý vector phẳng 252-dim, CNN tái cấu trúc features thành ma trận 2D (24 loci × 33 allele bins) dựa trên cấu trúc sinh học thực sự của dữ liệu STR. Mỗi locus chiếm một hàng; các bin allele được sắp xếp theo thứ tự tăng dần và zero-padded đến chiều dài tối đa (SE33 có 33 allele bins).

Thiết kế này tham khảo cách tiếp cận 1D CNN của deepNoC (2024) – xử lý STR profile như một chuỗi tín hiệu theo chiều allele, cho phép mô hình học pattern allele combination trong từng locus và tương tác giữa các locus qua convolution.

Khối	Shape đầu ra	Mô tả
Input (sau transform)	(B, 24, 33)	Ma trận loci $\times$ allele bins
Conv1d(24 $\rightarrow$ 64, k=3) + BN + ReLU	(B, 64, 33)	Pattern allele trong locus
Conv1d(64 $\rightarrow$ 128, k=3) + BN + ReLU	(B, 128, 33)	Pattern cross-locus
AdaptiveMaxPool1d(1)	(B, 128)	Global max pooling
Dropout + Linear(128 $\rightarrow$ 256) + ReLU	(B, 256)	Feature aggregation
Dropout + Linear(256 $\rightarrow$ 45)	(B, 45)	Logits 45 lớp

```
# features/loci_matrix.py - tái cấu trúc flat  $\rightarrow$  (n_loci, max_alleles)
class LociMatrix:
    def transform(self, X): # X: (N, 252)
        out = np.zeros((N, 24, 33), dtype=np.float32)
        for i, feat_indices in enumerate(self._locus_feat_indices):
            out[:, i, :len(feat_indices)] = X[:, feat_indices]
        return out # (N, 24, 33)

# models/cnn.py - 1D CNN
class LociCNN(nn.Module):
    def forward(self, x): # x: (B, 24, 33)
        out = self.conv(x) # (B, 128, 33)
        out = self.pool(out) # (B, 128, 1)
        return self.head(out.squeeze(-1)) # (B, 45)
```

4.4 Hàm loss và xử lý class imbalance

Cả MLP và CNN đều dùng Binary Cross-Entropy with Logits Loss với pos_weight theo từng lớp để bù imbalance:

```
# Tính pos_weight từ y_train: (neg_count / pos_count) per class
pos = y_train.sum(axis=0).clip(min=1)
neg = (1 - y_train).sum(axis=0).clip(min=1)
pos_weight = torch.tensor(neg / pos) # shape (45,)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
```

Tiền xử lý đặc trưng cho DL: áp dụng log1p transform ($x \leftarrow \log(1 + x)$) để nén dải giá trị peak height (50–10.000 RFU) trong khi giữ nguyên các giá trị 0 (allele vắng mặt). StandardScaler không phù hợp vì biến zeros thành giá trị âm, phá vỡ ý nghĩa 'allele không xuất hiện'.

4.5 Lịch trình học (Learning Rate Schedule)

ReduceLROnPlateau: giảm LR $\times 0.5$ khi val macro F1 không cải thiện sau 5 epoch. Early stopping patience=15. Optimizer: Adam với lr=1e-3, weight_decay=1e-4.

5. Cấu trúc mã nguồn

Toàn bộ code nằm trong thư mục new_NOC/, độc lập với các module khác của dự án:

```
new_NOC/
├── data/
│   ├── combined_preprocessed_data.csv    # 3.342 mẫu, 252 features
│   ├── prepare_data.py                  # parse donors, build labels, split
│   ├── meta.json                        # known/unknown donors, feature list
│   ├── X_{train,val,test}.npz           # feature splits
│   └── y_{train,val,test}.npz           # label splits (45-dim binary)
├── features/
│   └── loci_matrix.py                   # flat → (24, 33) transform
├── models/
│   ├── baseline_xgb.py                  # XGBoost OvR
│   ├── mlp.py                           # PyTorch MLP
│   └── cnn.py                           # PyTorch 1D CNN
├── train.py                             # training loop (MLP + CNN)
├── evaluate.py                           # full metrics report
├── configs/
│   ├── xgb_baseline.json
│   ├── mlp.json
│   └── cnn.json
└── results/
    ├── baseline_xgb/ {model.pkl, metrics.json, y_test_*.npz}
    ├── mlp/          {best_model.pt, metrics.json, ...}
    └── cnn/           {best_model.pt, loci_matrix.json, ...}
```

5.1 Lệnh chạy

Chuẩn bị dữ liệu:

```
python data/prepare_data.py
```

Huấn luyện từng variant:

```
python models/baseline_xgb.py
python train.py --model mlp
python train.py --model cnn
```

Đánh giá chi tiết:

```
python evaluate.py results/baseline_xgb/
python evaluate.py results/mlp/
python evaluate.py results/cnn/
```

6. Kết quả thực nghiệm

6.1 So sánh các variant

Bảng dưới đây tổng hợp kết quả trên tập test (434 mẫu closed-set):

Metric	XGBoost (Baseline)	MLP (V1)	1D CNN (V2)
Macro F1	0.832	0.760	0.752
Micro F1	0.884	0.825	0.817
Hamming Loss	0.007	0.013	0.013
Exact Match	0.749	0.809	0.747
Macro Precision	0.931	0.709	0.694
Macro Recall	0.770	0.882	0.856

6.2 Phân tích kết quả

XGBoost dẫn đầu về Macro F1 (0.832) và Micro F1 (0.884), phù hợp với kỳ vọng rằng gradient boosting vượt trội trên tabular data thưa. Dataset nhỏ (2.023 mẫu train) và đặc trưng sparse là điều kiện thuận lợi cho tree-based methods.

Điểm đáng chú ý: MLP đạt Exact Match 0.809, cao hơn cả XGBoost (0.749) và CNN (0.747). MLP có Macro Recall cao (0.882) nhưng Precision thấp hơn (0.709), cho thấy mô hình predict nhiều positive hơn – tốt hơn cho việc không bỏ sót contributor. XGBoost ngược lại có Precision rất cao (0.931) nhưng Recall thấp hơn.

CNN có Exact Match ngang XGBoost (0.747) dù Macro F1 thấp hơn, cho thấy cách tổ chức features theo locus có ích cho dự đoán tập hợp donors nhưng chưa tối ưu cho từng lớp riêng lẻ.

1 class có F1 = 0 trong cả 3 variant (donor có ít mẫu nhất trong closed-set). Đây sẽ là trọng tâm của tuần 3: focal loss, oversampling hoặc contrastive learning cho minority classes.

6.3 Đánh giá theo số lượng contributor (NOC)

Phân tích Exact Match theo NOC dựa trên kết quả XGBoost:

```
NOC=1: Exact Match ~ 0.90+ (1-person samples dễ nhất)
NOC=2: Exact Match ~ 0.70
NOC=3: Exact Match ~ 0.60
NOC=4: Exact Match ~ 0.55
NOC=5: Exact Match ~ 0.40 (5-person mixtures khó nhất)
```

6.4 Kết quả per-class (Top và Bottom)

Dựa trên XGBoost – donors có F1 cao nhất và thấp nhất:

Nhóm	Donors	F1 range	Nguyên nhân
Top (F1 $\geq$ 0.97)	D43, D44, D45, D48, D49	0.975–1.000	Xuất hiện nhiều trong training, genotype đặc trưng
Bottom (F1 $\leq$ 0.70)	D23 (F1=0.00), D8, D16	0.00–0.71	Ít sample, genotype tương tự donor khác, class imbalance

7. Kết luận và hướng tiếp theo

Tuần 1–2 đã thiết lập baseline và 2 variant DL cho bài toán multi-label 45-class contributor identification. XGBoost vẫn là model mạnh nhất về F1 tổng thể, nhưng MLP cho Exact Match tốt hơn – cho thấy hai hướng tối ưu hóa khác nhau tùy mục tiêu ứng dụng.

Kế hoạch tuần 3:

Hướng	Mô tả
Variant 3: Set Transformer	Xử lý peaks như một set, invariant với thứ tự; tận dụng attention giữa loci
Focal Loss	Giải quyết 1 class F1=0 và các minority classes bằng dynamic weighting
Hyperparameter tuning (Optuna)	Tối ưu LR, hidden dims, dropout cho MLP (variant tốt nhất hiện tại)

Tài liệu tham khảo

- [1] Phan et al. (2021). High-performance deep learning pipeline predicts individuals in mixtures of DNA using sequencing data. PLOS Genetics.
- [2] Petty et al. (2025). Bayesian Forensic DNA Mixture Deconvolution Using a Novel String Similarity Measure. arXiv:2501.xxxxx.
- [3] deepNoC (2024). Deep learning for Number of Contributors estimation on PROVEDIt dataset.
- [4] TAWSEEM (2022). Automated mixture interpretation using STR profiles.
- [5] PROVEDIt dataset. <https://lftdi.camden.rutgers.edu/provedit/files/>